

Project #1:

clemsh: a self-analyzing shell

Objectives

In this project, you will learn about shells, processes, process control, pipes, sockets, file descriptors, environment variables, and shims. Your mission is to build a simple command shell that runs programs; supports redirection to files, network end points, and other processes; and logs information about what it and its child processes are doing.

A Simple Shell

A shell is simply a program that processes commands, runs other processes, and returns input. If you are using a terminal on your computer, you are using a shell. Common shells that you may have used before include **bash** and **zsh** (or PowerShell on Windows). In this assignment we're going to write our own shell.

Your shell will be called **clemsh**. It will be run from the command-line with the syntax shown in Listing 1. It can be run in two modes: interactive mode and batch mode.

Modes: When **clemsh** is run without any arguments, it should run in **interactive mode**. In this mode, the program will print a prompt ("**>** ") and allow the user to type a command pipeline (one or more commands piped together with that pipe symbol, **|**). When the program gets a newline (**\n**) char-

Listing 1: Syntax for running your shell.

```
clemsh [<command file>]  
Interactive Mode: ./clemsh  
> ls  
clemsh.c      Makefile      envshim.c  
> ls | wc  
      13      13      133  
Batch Mode: ./clemsh commands.txt
```

Listing 2: Command Syntax and Examples.**Supported Examples**

```
> ls
> ls | wc >out.txt
> ./test1
> cat <input.txt | wc <tcp:localhost:8080 >tcp:localhost
:9090
> ./test1 2 4 foo | ./test2 "hello world" > myfile.txt
> ./test1 2 4 foo | ./test2 17
```

Examples you don't have to support

```
> ls -a ; ls -l
> ls | wc >out.txt >out2.txt
```

acter (when the user presses enter) your shell should execute the pipeline and print its output.

When **clemsh** is run with a single argument (**batch mode**), that argument should specify a valid filename that contains a list of command pipelines to run, one pipeline per line. Your shell will read the list of pipelines, execute each line in the order they are listed, and print out the output of the commands. In batch mode, your shell should not print out a prompt or the commands being run, just their output.

Command Syntax: Your shell accepts a simple set of command pipelines, examples shown in Listing 2. A *pipeline* is made up of one or more *commands* or *process invocations* separated by the pipe character (`|`), which directs the shell to connect the left process's **stdout** to the right process's **stdin**. Your shell should also support file and network redirections, indicated by either the greater-than character (`>`) or the less-than character (`<`) to connect the process's **stdout** or **stdin** to a file or network end-point. Files are specified simply by a filename. Network end-points should have the form **tcp:host:port**, where *host* is a valid network host (like **localhost**, **127.0.0.1**, **google.com**) and *port* is a valid TCP port number. For this project, we will only support TCP end-points.

In this project, you don't have to worry about redirecting **stderr** or duplicate redirections (like **ls >out1.txt >out2.txt**). To simplify things a bit, and since I don't want you spending all of your time on parsing, I have provided you with a parser in the class git repository. You are allowed to use this parser code in your submission.

For each *command*, you should run the process with the provided arguments using **fork** and **execvp**. For redirection, you should connect processes together using the **pipe** and **dup2** functions. You will also need to open files

(using either **open** or **fopen** to get a file descriptor) and sockets (using the **getaddrinfo**, **socket**, and **connect** functions).

See the class git repository for examples on how to use these functions.

The multiple stages of a pipeline should run concurrently (in separate child processes). Pipelines should run sequentially (one pipeline must complete before the next one begins).

Handling multiple input/output options: One interesting case you will run into in this project is a pipeline that provides conflicting redirections. For example, in the pipeline `"./hello | wc <in.txt"`, **wc** is offered two inputs—the piped input from **hello** and the file input from **in.txt**.

Different shells handle this differently. We are going to use the approach used by **bash**, which simply gives file (and in our case network) redirects precedence. So, the previously mentioned pipeline, the `"<in.txt"` redirect would override the piped input from `./hello`, and **wc** would receive just the contents of **in.txt**. This applies to both file output redirects (in `"./hello >out.txt | wc"` the content produced by `./hello` should end up in the file, **out.txt**, rather than being piped to **wc**) and network redirects (for `"./hello >tcp:localhost:8070 | wc"`, all output from `./hello` goes to the network target, not **wc**).

The Self-Analyzing Part: One useful thing about writing your own shell is that you can have it collect information about the processes, commands, and pipelines that it runs. This can be useful for performance testing, debugging, and just understanding how processes behave. For this assignment, your shell is going to track a simple set of data—which processes are executed, when those processes exit (and with what status), and what environment variables the processes read and modify.

When your shell runs, if the **CLEMSHLOG** environment variable is set, your shell should also produce a log file, named **clemshlog.txt**, in the current working directory. If the **CLEMSHLOG** environment variable is set, this log file should be overwritten every time your shell runs in either interactive or batch mode.

The log file format is shown in Listing 3. Each line represents an event that occurred. Specifically, the events you will log are **START** (a process started—you report the command that was run), **EXIT** (a process exited—you report the clock/system/user time it took to run (in ms), its exit status if it finished normally, and if it crashed due to a signal report the signal number), **ENVREAD** (a process called `getenv` to read an environment value—you report the variable and the value that was read), and **ENVWRITE** (a process called `setenv` or `putenv` to an environment variable to a new value—you report the variable and the new value assigned to it). Each event line should start with the name

Listing 3: The clemsh Log File Format**General Log Format**

```
EVENT<TAB>PIPELINE : STAGE<TAB>[EVENT DETAILS]
```

Specific Event Formats

```
START<TAB>PIPELINE : STAGE<TAB><COMMAND>
```

```
EXIT<TAB>PIPELINE : STAGE<TAB><REALTIMEinMS><TAB><
```

```
    SYSTIMEinMS><TAB><USERTIMEinMS><TAB>NORMAL<TAB><TAB><
    STATUS>
```

```
EXIT<TAB>PIPELINE : STAGE<TAB><REALTIMEinMS><TAB><
```

```
    SYSTIMEinMS><TAB><USERTIMEinMS><TAB>CRASH<TAB><TAB><
    SIGNAL>
```

```
ENVREAD<TAB>PIPELINE : STAGE<TAB><VAR>=<VALUE>
```

```
ENVWRITE<TAB>PIPELINE : STAGE<TAB><VAR>=<VALUE>
```

Log Example

```
START      0:0    ls -al -h
ENVREAD    0:0    HOME=/home/user
START      0:1    wc -l
ENVREAD    0:1    PATH=/bin:/usr/local/bin
ENVWRITE   0:0    DEBUG=1
ENVWRITE   0:0    DEBUG=
EXIT       0:0    1.608875    0.843000    0.0000    NORMAL    0
EXIT       0:1    3.20811    1.243000    0.0000    NORMAL    0
START      1:0    ./buggy
EXIT       1:0    9.608875    2.843000    0.0000    CRASH     11
```

of the event, and the pipeline number and stage number of the process (start counting pipelines and stages at zero).

Some of the log events (**START**, **EXIT**) can be gathered directly by the shell process. Others (**ENVREAD**, **ENVWRITE**) will require the use of a shim library (you will call it **envshim.so**) to intercept calls to **getenv**, **setenv**, **putenv**, and **unsetenv**. You should ignore any other methods for accessing environment variables.

Your shim library can be implemented as a single C file (you can of course use more than one file if you like), that implements at least the four functions shown in Listing 4. Your shim will have to communicate (using some form of interprocess communication — your choice) with the shell in order to match up processes with their pipeline and stage. Log file entries can be written by either the shim or your shell process. As long as they end up in the log file, the autograder does not care who put them there. Your shim code should also not produce any output to stderr or stdout (debug statements will likely confuse the autograder).

Listing 4: Hints for your environment variable intercepting shim.

```
//implement these functions
char * getenv (const char *name);
int setenv (const char *name, const char *value, int
            overwrite);
int putenv (char *string);
int unsetenv (const char *name);

//how to call the original version of getenv
#include <dlfcn.h>
char *(*original_getenv) (const char *);
original_getenv = dlsym(RTLD_NEXT, "getenv");
char *value = original_getenv("PATH");
```

Also, keep in mind that the processes in a pipeline should run concurrently. So, your START and EXIT events may not always occur in the same order.

There are multiple ways to implement this program. You can use any of the communication methods we discussed in class (files, pipes, sockets, etc). It's up to you.

For timing, you can use `clock_gettime` or `timespec_get` to measure real-time (wall clock time). For system and user time, you can use `getrusage` or `wait3` or `wait4`. Detecting how a process exited can be done using the `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, and `WTERMSIG` macros. These are described in the man pages for `wait` and `waitpid`.

Also, note that there are several things that the child process can do that can interfere with your shim. A program can terminate abnormally (by calling `abort`) which will prevent library destructors from running.

What to do about errors?

No matter what I throw at you, I should not be able to crash your shell, and I will try. It should always exit with either status 0 (`EXIT_SUCCESS`) or 130 (`SIGINT` — when the user exits interactive mode by pressing Ctrl-C). It should never abort, seg fault, or crash regardless of what commands I pass into it. I do have my limits. I am not going to run multi-process applications in your shell. I am also not going to send signals to your shell (other than `SIGINT`), but anything else is fair game.

If you run into an error, you should print out a reasonable 1-line error mes-

sage that describes what went wrong (to **stderr**). The parser will provide you with error messages for malformed inputs that don't follow the guidelines. You are only expected to execute pipelines up to 1024 characters long. Any input longer than that should be considered to be an error. Don't run the overlong pipeline. Just print out an error message and move on to the next pipeline. Other errors could include (but are not limited to) programs that don't exist or crash, redirection to/from files that don't exist, or network redirection errors.

In the interest of keeping things sane and reducing your debugging time, make your error messages descriptive. Tell us what happened. Consider using **errno** and **strerror**. Also, as you start dealing with more involved test cases, I recommend including the pipeline number and (when applicable) the stage to help you narrow down where the message is coming from.

Other thoughts, hints, and warnings

Your program should be written in C and use good programming style. Your code should be **-Wall**¹ clean, and most importantly it should be your own. Your shell will be tested on single-process terminal programs. You don't need to correctly support multiprocess applications (applications, like Google Chrome that create more processes).

For this project, there are several tools that will be helpful. The first is **gdb**. I'm often surprised at how many students don't use **gdb**, which I think makes life much easier. I strongly recommend it. If you haven't before, I also recommend that you become familiar with **valgrind**, a tool that can help you find memory leaks and other memory errors. For testing network redirects, I also find **nc** (netcat) to be very helpful.

Make sure that your code compiles, runs and has been thoroughly tested on the lab machines.

Submission Instructions

This project has two deadlines. Absolutely no late assignments will be accepted, and deadline extensions typically require a documented medical emergency or natural disaster.

¹**“-Wall clean”** = If compiled with the **-Wall** flag, it should not produce any warnings.

Compile deadline: 4:00 PM, September 4th — This is a preliminary deliverable worth 10 points. For this, you simply need to submit a valid `tgz` file (more info below on this) with code and a Makefile that compiles without any errors or warnings and produces two files, called **clemsh** and **envshim.so**. The compiled program and library do not need to work correctly. For this deliverable, submit to the **compile1** assignment on Canvas.

Functional deadline: 4:00 PM, September 16th — This is the final submission deadline worth the remaining 60 points. For this deliverable, submit to the **project1** assignment on Canvas.

Creating your submission: When your project is complete, archive your source materials, using the following command:

```
> tar cvzf project1.tgz README Makefile <list of source files>
```

Your **Makefile** should build your program (when I run **make**). It should produce two files, called **clemsh**, and **envshim.so**.

The **README** file should include your name, a short description of your project, and any other comments you think are relevant to the grading. It should have two clearly labeled sections titled with KNOWN PROBLEMS, and DESIGN. The KNOWN PROBLEMS section should include a specific description of all known problems or program limitations. This will not hurt your grade and may (in rare cases) improve it. I am more sympathetic when students have thoroughly tested their code and clearly understand their shortcomings, and less so when I find bugs that students are ignorant of or have tried to hide. You should also include references to any materials that you referred to while working on the project. Please do not include special instructions for compilation. The compilation and running of the tests will be automated (see details below), and your instructions will not be followed.

Please make sure you include only the source files, not the object files or compiled binaries. Also, make sure that your files are not in a subdirectory within the tar ball. Before you submit this single file, please use the following commands to check and make sure you have everything in the **project1.tgz** file:

```
> mkdir test; cd test
> tar xvzf <path to tgz file>/project1.tgz
> make
```

This should put all of your source files in the current directory, and compile your program and shim.

Submit your **project1.tgz** file via **Canvas**. You must name your archive **project1.tgz**, and it must compile and run on the McAdams Lab machines.

We will compile your code using your **Makefile**, and run it using the syntax described in Listing 1.

1 Grading

Your project will be graded based on the results of functional testing and the design of your code. We will run several tests to make sure it works properly and correctly handles various error conditions. We will test your programs against custom test programs that we have written as well as common terminal programs. I will provide a few simple testcases in the class git repository, but you will not have access to the autograder testcases that are used to grade your project. Your code should not crash. You will receive 10% credit if your code successfully compiles, and 10% for code style and readability. The rest of your score will be determined by the number of tests that your shell passes.

Your source materials should be readable, properly documented and use good coding practices (avoid magic numbers, use appropriately-named variables, use good indentation, etc). Your code should be -Wall clean (you should not have any warnings during compilation). **Our testing of your code will be thorough. Be sure you test your application thoroughly.**

Reminders: You can submit to Canvas multiple times before the deadline. Your final submission will be the one that counts even if you have a previous submission with a higher score. The autograder will run once a day. Starting early means you can get more autograder attempts.

2 Collaboration/Integrity

You will work independently on this project. You must *not* discuss your solution with classmates, and all of your code must be your own code (no AI-generated or borrowed/stolen code), produced by your own brain. If the code generation capabilities in your editor are too tempting, consider using something like Vim.

You may, of course, discuss the project with me, and you may discuss conceptual issues related to the project that we have already discussed in lecture or on Piazza. For example, it would be OK to discuss how **waitpid** works (in general) since this is something we have discussed in class. It is NOT OK to discuss how you specifically used **waitpid** to solve any part of this project.

Collaborating with peers on this project, in this or any other manner will be treated as academic misconduct.

Finally, access to the autograder is a privilege that can be revoked. Please refer to the course syllabus for information about my autograder use policies.